

Single-timestep PNG NMR workflow demo

This notebook demonstrates one complete NMR workflow for one RTSPHEM timestep:

1. Geometry import and bbox size check
2. Editable parameter block
3. PNG pixel decay solve
4. PNG triangular-mesh decay solve and mesh preview
5. COMSOL / PNG pixel / PNG triangular decay and T2 inversion comparison
6. Timing summary

Every run creates a timestamped output folder under the selected experiment directory.

1. Parameter entry

Edit this cell when switching experiment folders, timestep, or numerical/physical parameters. The defaults below run `exp_015`, `timestep=15`.

```
In [9]: from pathlib import Path
from datetime import datetime

# -----
# Input path and timestep
# -----
EXP_DIR = Path(r"C:\Users\imgw\Documents\Codex\RTSPHEM-main\outputs\rtm_batches\33\
TIMESTEP = 25

# Project directory. The unicode escapes keep this notebook runnable with
# `jupyter run` on Windows consoles that default to cp1252.
PROJECT_DIR = Path("C:/Users/imgw/Documents/Codex/NMR\u6a21\u62df")
ADVANCED_TOOLS_DIR = PROJECT_DIR / "advanced_tools"
PNG_DECAY_SCRIPT = ADVANCED_TOOLS_DIR / "png_phase_nmr_decay.py"
PYGIMLI_FALLBACK_PYTHON = Path(r"C:\Python314\python.exe")

# -----
# Geometry interpretation
# -----
USE_NONWHITE_BBOX_FOR_GEOMETRY_SIZE = True
MAX_GRID_SIZE = 500

# -----
# Physical parameters
# -----
DIFFUSION_UM2_PER_MS = 2.0
```

```

BULK_T2_MS = 3000.0
RHO_SOLID_UM_PER_MS = 0.05
RHO_GAS_UM_PER_MS = 0.0

# -----
# Decay solver parameters
# -----
DECAY_DT_MS = 5.0
DECAY_T_MAX_MS = 5500.0

# -----
# Triangular mesh parameters
# -----
TRI_MESH_BULK_SIZE_UM = 16.0
TRI_MESH_BOUNDARY_SIZE_UM = 5.0
TRI_MESH_MAX_POINTS = 25000

# -----
# T2 inversion parameters
# -----
NNLS_ALPHA = 0.1
T2_MIN_MS = 1.0
T2_MAX_MS = 100000.0
T2_NUM_BINS = 240

# -----
# Output directory. Each run is separated by timestamp.
# -----
RUN_ID = f"single_timestep_nmr_demo_t{TIMESTEP:04d}_" + datetime.now().strftime("%Y
RUN_DIR = EXP_DIR / RUN_ID
RUN_DIR.mkdir(parents=True, exist_ok=False)

INTERFACE_DIR = EXP_DIR / "interface_images"
COMSOL_DIR = EXP_DIR / "comsol_results"
CONCENTRATION_DIR = EXP_DIR / "individual_plots" / "concentration"
METADATA_JSON = EXP_DIR / "run_metadata.json"
TIME_CSV = EXP_DIR / "global_evolution_log.csv"

PNG_PATH = INTERFACE_DIR / f"timestep_{TIMESTEP:04d}.png"
COMSOL_XLSX = COMSOL_DIR / f"T2_t{TIMESTEP:04d}.xlsx"
CONCENTRATION_PNG = CONCENTRATION_DIR / f"concentration_{TIMESTEP:04d}.png"

print("RUN_DIR:", RUN_DIR)
print("PNG:", PNG_PATH)
print("COMSOL:", COMSOL_XLSX)
print("Concentration:", CONCENTRATION_PNG)

```

```

RUN_DIR: C:\Users\imgw\Documents\Codex\RTSPHEM-main\outputs\rtm_batches\33\exp_015\s
ingle_timestep_nmr_demo_t0025_20260528_155335
PNG: C:\Users\imgw\Documents\Codex\RTSPHEM-main\outputs\rtm_batches\33\exp_015\inter
face_images\timestep_0025.png
COMSOL: C:\Users\imgw\Documents\Codex\RTSPHEM-main\outputs\rtm_batches\33\exp_015\co
msol_results\T2_t0025.xlsx
Concentration: C:\Users\imgw\Documents\Codex\RTSPHEM-main\outputs\rtm_batches\33\exp
_015\individual_plots\concentration\concentration_0025.png

```

2. Imports and helper setup

```
In [10]: import csv
import json
import os
import subprocess
import sys
import time

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from IPython.display import Image as IPyImage, Markdown, display
from PIL import Image

sys.path.insert(0, str(PROJECT_DIR))

from advanced_tools.compare_png_nmr_with_comsol import (
    invert_fixed_alpha,
    normalize,
    read_comsol_probe_excel,
    read_png_decay_csv,
    trim_to_decay,
)
from advanced_tools.png_phase_nmr_decay import (
    OUTSIDE,
    SimulationParams,
    classify_png,
    downsample_nearest,
    sample_bbox,
    simulate_png,
)

try:
    import pygimli as pg
    CURRENT_PYTHON_HAS_PYGIMLI = True
except Exception:
    CURRENT_PYTHON_HAS_PYGIMLI = False

for path in [EXP_DIR, PNG_PATH, COMSOL_XLSX, METADATA_JSON]:
    if not path.exists():
        raise FileNotFoundError(path)

print("Current Python:", sys.executable)
print("Current Python has pyGIMLi:", CURRENT_PYTHON_HAS_PYGIMLI)
print("Fallback pyGIMLi Python:", PYGIMLI_FALLBACK_PYTHON)
```

Current Python: c:\Users\imgw\.conda\envs\ml\python.exe

Current Python has pyGIMLi: False

Fallback pyGIMLi Python: C:\Python314\python.exe

3. Geometry import and size check

```

In [11]: metadata = json.loads(METADATA_JSON.read_text(encoding="utf-8"))
params = metadata["parameters"]
length_x_um = float(params["lengthXAxis_cm"]) * 10000.0
length_y_um = float(params["lengthYAxis_cm"]) * 10000.0

rgb = np.asarray(Image.open(PNG_PATH).convert("RGB"))
labels = classify_png(rgb)
raw_height, raw_width = labels.shape

if USE_NONWHITE_BBOX_FOR_GEOMETRY_SIZE:
    r_min, r_max, c_min, c_max = sample_bbox(labels)
    bbox_height = r_max - r_min + 1
    bbox_width = c_max - c_min + 1
    pixel_size_x_um = length_x_um / bbox_width
    pixel_size_y_um = length_y_um / bbox_height
    geometry_mode = "nonwhite bbox"
else:
    r_min, r_max, c_min, c_max = 0, raw_height - 1, 0, raw_width - 1
    bbox_height = raw_height
    bbox_width = raw_width
    pixel_size_x_um = length_x_um / raw_width
    pixel_size_y_um = length_y_um / raw_height
    geometry_mode = "full PNG"

geometry_dir = RUN_DIR / "01_geometry"
geometry_dir.mkdir(parents=True, exist_ok=True)
fig, ax = plt.subplots(figsize=(8, 5))
ax.imshow(rgb)
rect = plt.Rectangle((c_min, r_min), bbox_width, bbox_height, fill=False, color="cy")
ax.add_patch(rect)
ax.set_title(f"Geometry import: {geometry_mode}")
ax.set_axis_off()
geometry_png = geometry_dir / f"timestep_{TIMESTEP:04d}_geometry_bbox.png"
fig.savefig(geometry_png, dpi=220, bbox_inches="tight")
plt.show()

geometry_summary = {
    "timestep": TIMESTEP,
    "geometry_mode": geometry_mode,
    "length_x_um": length_x_um,
    "length_y_um": length_y_um,
    "raw_width_px": raw_width,
    "raw_height_px": raw_height,
    "bbox_col_min": c_min,
    "bbox_col_max": c_max,
    "bbox_row_min": r_min,
    "bbox_row_max": r_max,
    "bbox_width_px": bbox_width,
    "bbox_height_px": bbox_height,
    "pixel_size_x_um": pixel_size_x_um,
    "pixel_size_y_um": pixel_size_y_um,
    "outside_px": int(np.sum(labels == 0)),
    "water_px": int(np.sum(labels == 1)),
    "solid_px": int(np.sum(labels == 2)),
}

```

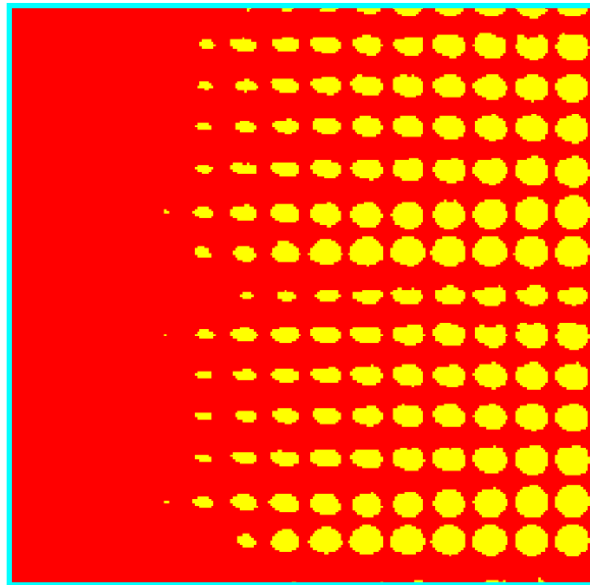
```
pd.DataFrame([geometry_summary]).to_csv(geometry_dir / "geometry_summary.csv", index=False)
display(pd.DataFrame([geometry_summary]))
display(IPyImage(filename=str(geometry_png)))
```

C:\Users\imgw\AppData\Local\Temp\ipykernel_60280\1408694117.py:35: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
plt.show()

	timestep	geometry_mode	length_x_um	length_y_um	raw_width_px	raw_height_px
0	25	nonwhite bbox	995.0	980.0	1343	808



Geometry import: nonwhite bbox



4. Parameter table

```
In [12]: parameter_table = pd.DataFrame([
    {"name": "DIFFUSION_UM2_PER_MS", "value": DIFFUSION_UM2_PER_MS, "note": "D, COM"},
    {"name": "BULK_T2_MS", "value": BULK_T2_MS, "note": "bulk T2"},
    {"name": "RHO_SOLID_UM_PER_MS", "value": RHO_SOLID_UM_PER_MS, "note": "solid-li"},
    {"name": "RHO_GAS_UM_PER_MS", "value": RHO_GAS_UM_PER_MS, "note": "gas-liquid s"},
    {"name": "DECAY_DT_MS", "value": DECAY_DT_MS, "note": "decay time step"},
    {"name": "DECAY_T_MAX_MS", "value": DECAY_T_MAX_MS, "note": "maximum decay time"},
    {"name": "MAX_GRID_SIZE", "value": MAX_GRID_SIZE, "note": "largest image dimens"},
    {"name": "TRI_MESH_BULK_SIZE_UM", "value": TRI_MESH_BULK_SIZE_UM, "note": "tria"},
    {"name": "TRI_MESH_BOUNDARY_SIZE_UM", "value": TRI_MESH_BOUNDARY_SIZE_UM, "note": "tri"},
    {"name": "NNLS_ALPHA", "value": NNLS_ALPHA, "note": "fixed T2 inversion smoothi"},
    {"name": "T2_MIN_MS", "value": T2_MIN_MS, "note": "T2 inversion lower bound"},
    {"name": "T2_MAX_MS", "value": T2_MAX_MS, "note": "T2 inversion upper bound"},
    {"name": "T2_NUM_BINS", "value": T2_NUM_BINS, "note": "number of T2 bins"},
])
```

```
parameter_table.to_csv(RUN_DIR / "parameter_table.csv", index=False)
display(parameter_table)
```

	name	value	note
0	DIFFUSION_UM2_PER_MS	2.00	D, COMSOL $2\text{e-}9\text{ m}^2/\text{s} = 2\text{ um}^2/\text{ms}$
1	BULK_T2_MS	3000.00	bulk T2
2	RHO_SOLID_UM_PER_MS	0.05	solid-liquid surface relaxivity
3	RHO_GAS_UM_PER_MS	0.00	gas-liquid surface relaxivity
4	DECAY_DT_MS	5.00	decay time step
5	DECAY_T_MAX_MS	5500.00	maximum decay time
6	MAX_GRID_SIZE	500.00	largest image dimension after downsampling
7	TRI_MESH_BULK_SIZE_UM	16.00	triangular mesh interior spacing
8	TRI_MESH_BOUNDARY_SIZE_UM	5.00	triangular mesh boundary spacing
9	NNLS_ALPHA	0.10	fixed T2 inversion smoothing factor
10	T2_MIN_MS	1.00	T2 inversion lower bound
11	T2_MAX_MS	100000.00	T2 inversion upper bound
12	T2_NUM_BINS	240.00	number of T2 bins

5. PNG pixel decay solve

```
In [13]: pixel_dir = RUN_DIR / "02_png_pixel_decay"
pixel_dir.mkdir(parents=True, exist_ok=True)

pixel_params = SimulationParams(
    pixel_size_x_um=pixel_size_x_um,
    pixel_size_y_um=pixel_size_y_um,
    diffusion_um2_per_ms=DIFFUSION_UM2_PER_MS,
    bulk_t2_ms=BULK_T2_MS,
    rho_solid_um_per_ms=RHO_SOLID_UM_PER_MS,
    rho_gas_um_per_ms=RHO_GAS_UM_PER_MS,
    dt_ms=DECAY_DT_MS,
    t_max_ms=DECAY_T_MAX_MS,
    max_grid_size=MAX_GRID_SIZE,
    solver="pixel",
    mesh_bulk_size_um=TRI_MESH_BULK_SIZE_UM,
    mesh_boundary_size_um=TRI_MESH_BOUNDARY_SIZE_UM,
    mesh_max_points=TRI_MESH_MAX_POINTS,
)

start = time.perf_counter()
pixel_summary = simulate_png(PNG_PATH, pixel_dir, pixel_params)
pixel_wall_time_s = time.perf_counter() - start
```

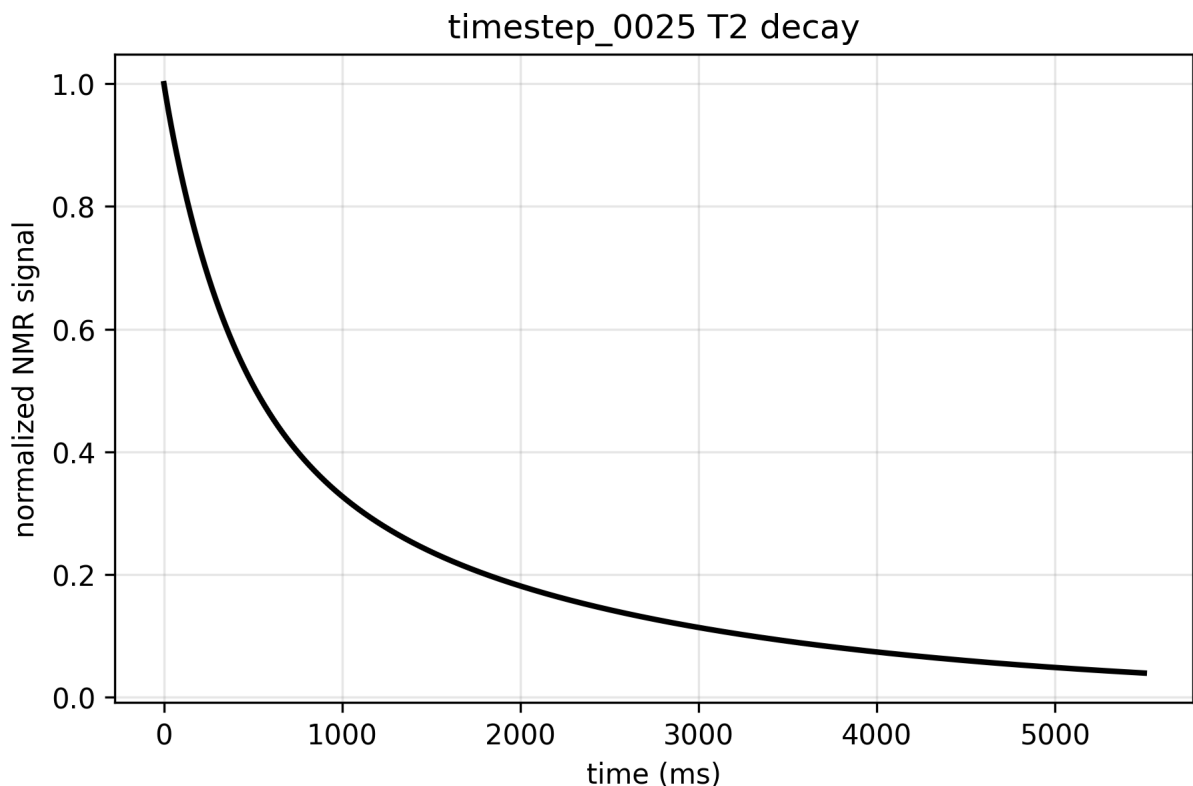
```

pixel_decay_csv = pixel_dir / f"timestep_{TIMESTEP:04d}_nmr_decay.csv"
pixel_decay_png = pixel_dir / f"timestep_{TIMESTEP:04d}_nmr_decay.png"
print(f"PNG pixel wall time: {pixel_wall_time_s:.3f} s")
display(pd.read_csv(pixel_decay_csv).head())
display(IPyImage(filename=str(pixel_decay_png)))

```

PNG pixel wall time: 5.845 s

	time_ms	signal	normalized_signal
0	0.0	775031.720934	1.000000
1	5.0	767870.799151	0.990760
2	10.0	760957.043904	0.981840
3	15.0	754237.398534	0.973170
4	20.0	747680.352682	0.964709



6. PNG triangular mesh solve and mesh preview

```

In [14]: tri_dir = RUN_DIR / "03_png_triangular_decay"
tri_dir.mkdir(parents=True, exist_ok=True)

tri_params = SimulationParams(
    pixel_size_x_um=pixel_size_x_um,
    pixel_size_y_um=pixel_size_y_um,
    diffusion_um2_per_ms=DIFFUSION_UM2_PER_MS,
    bulk_t2_ms=BULK_T2_MS,
    rho_solid_um_per_ms=RHO_SOLID_UM_PER_MS,
    rho_gas_um_per_ms=RHO_GAS_UM_PER_MS,

```

```

dt_ms=DECAY_DT_MS,
t_max_ms=DECAY_T_MAX_MS,
max_grid_size=MAX_GRID_SIZE,
solver="triangular",
mesh_bulk_size_um=TRI_MESH_BULK_SIZE_UM,
mesh_boundary_size_um=TRI_MESH_BOUNDARY_SIZE_UM,
mesh_max_points=TRI_MESH_MAX_POINTS,
)

start = time.perf_counter()
if CURRENT_PYTHON_HAS_PYGIMLI:
    tri_summary = simulate_png(PNG_PATH, tri_dir, tri_params)
else:
    cmd = [
        str(PYGIMLI_FALLBACK_PYTHON), str(PNG_DECAY_SCRIPT), str(PNG_PATH),
        "--output-dir", str(tri_dir),
        "--pixel-size-x-um", str(pixel_size_x_um),
        "--pixel-size-y-um", str(pixel_size_y_um),
        "--diffusion-um2-per-ms", str(DIFFUSION_UM2_PER_MS),
        "--bulk-t2-ms", str(BULK_T2_MS),
        "--rho-solid-um-per-ms", str(RHO_SOLID_UM_PER_MS),
        "--rho-gas-um-per-ms", str(RHO_GAS_UM_PER_MS),
        "--dt-ms", str(DECAY_DT_MS),
        "--t-max-ms", str(DECAY_T_MAX_MS),
        "--max-grid-size", str(MAX_GRID_SIZE),
        "--solver", "triangular",
        "--mesh-bulk-size-um", str(TRI_MESH_BULK_SIZE_UM),
        "--mesh-boundary-size-um", str(TRI_MESH_BOUNDARY_SIZE_UM),
        "--mesh-max-points", str(TRI_MESH_MAX_POINTS),
    ]
    env = dict(os.environ)
    env["PYTHONIOENCODING"] = "utf-8"
    env["MPLBACKEND"] = "Agg"
    completed = subprocess.run(
        cmd,
        check=False,
        capture_output=True,
        text=True,
        encoding="utf-8",
        errors="replace",
        env=env,
    )
    if completed.stdout:
        print(completed.stdout)
    if completed.returncode != 0:
        print("Subprocess command failed:")
        print(" ".join(str(x) for x in cmd))
        print("--- stdout ---")
        print(completed.stdout)
        print("--- stderr ---")
        print(completed.stderr)
        raise RuntimeError(f"Triangular mesh subprocess failed with return code {co
tri_wall_time_s = time.perf_counter() - start

tri_decay_csv = tri_dir / f"timestep_{TIMESTEP:04d}_nmr_decay.csv"
tri_decay_png = tri_dir / f"timestep_{TIMESTEP:04d}_nmr_decay.png"

```

```
tri_mesh_png = tri_dir / f"timestep_{TIMESTEP:04d}_triangular_mesh.png"
tri_quality_csv = tri_dir / f"timestep_{TIMESTEP:04d}_mesh_quality.csv"

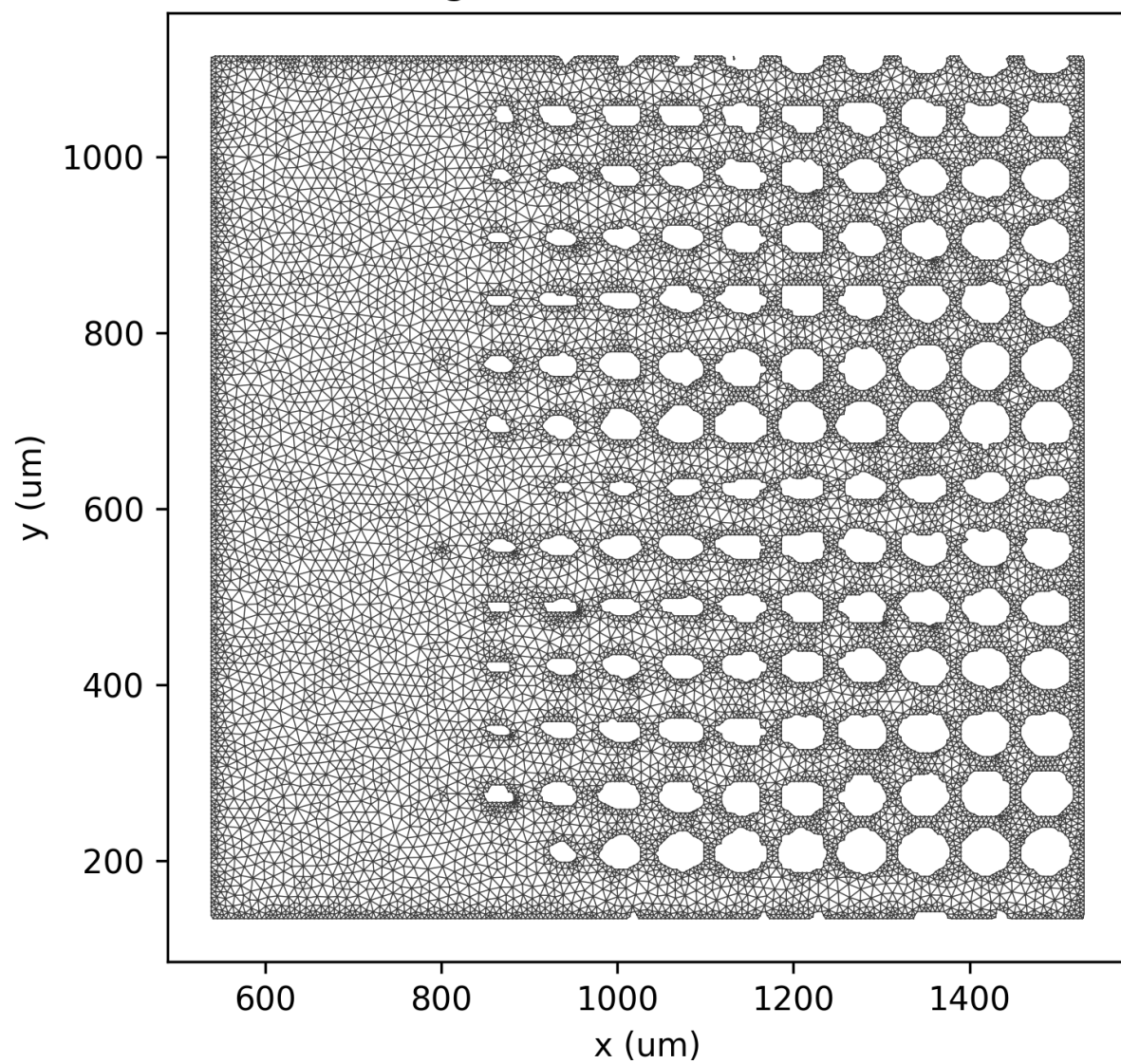
print(f"PNG triangular wall time: {tri_wall_time_s:.3f} s")
display(pd.read_csv(tri_decay_csv).head())
if tri_mesh_png.exists():
    display(IPyImage(filename=str(tri_mesh_png)))
display(IPyImage(filename=str(tri_decay_png)))
if tri_quality_csv.exists():
    display(pd.read_csv(tri_quality_csv).describe().T.head(10))
```

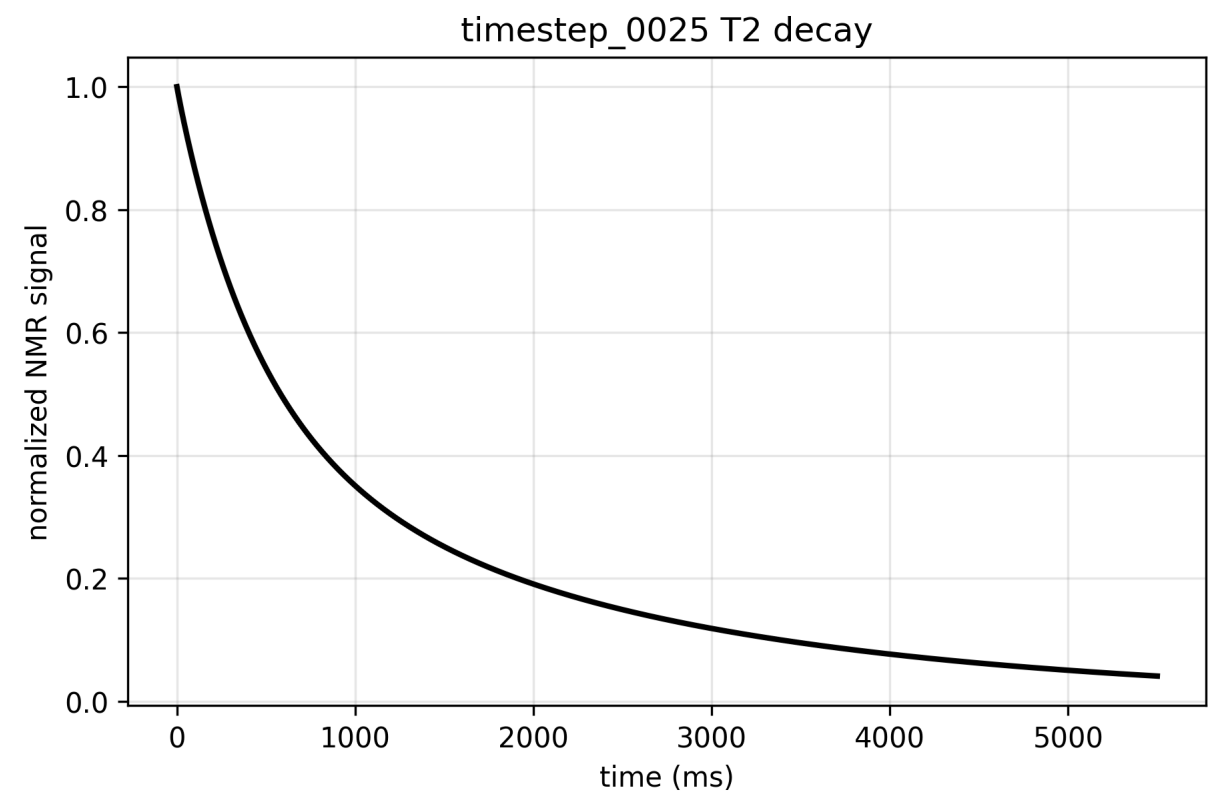
Processed 1 PNG file(s).
Outputs written to: C:\Users\imgw\Documents\Codex\RTSPHEM-main\outputs\rtm_batches\3
3\exp_015\single_timestep_nmr_demo_t0025_20260528_155335\03_png_triangular_decay

PNG triangular wall time: 5.203 s

	time_ms	signal	normalized_signal
0	0.0	777428.419401	1.000000
1	5.0	771270.942215	0.992080
2	10.0	765311.370672	0.984414
3	15.0	759502.298260	0.976942
4	20.0	753816.336682	0.969628

Triangular water-domain mesh





	count	mean	std	min	25
element_id	23913.0	11956.000000	6903.232830	0.000000	5978.000000
node_0	23913.0	7973.857651	3798.006549	0.000000	5247.000000
node_1	23913.0	8038.423075	3782.978621	0.000000	5360.000000
node_2	23913.0	8105.423619	3721.694772	1.000000	5473.000000
area_um2	23913.0	32.510702	22.959170	0.148021	15.127300
edge_min_um	23913.0	7.323793	2.757062	0.251122	5.000000
edge_max_um	23913.0	9.305975	3.121631	0.991325	6.913200
edge_mean_um	23913.0	8.346662	2.876023	0.761837	6.086000
quality_equilateral_normalized	23913.0	0.943545	0.056736	0.176922	0.920900
aspect_edge_ratio	23913.0	1.302105	0.214210	1.002421	1.160700



7. COMSOL / PNG pixel / PNG triangular T2 comparison

```
In [15]: comparison_dir = RUN_DIR / "04_comparison"
comparison_dir.mkdir(parents=True, exist_ok=True)

comsol_t_raw, comsol_s_raw = read_comsol_probe_excel(COMSOL_XLSX)
pixel_t_raw, pixel_s_raw = read_png_decay_csv(pixel_decay_csv)
```

```

tri_t_raw, tri_s_raw = read_png_decay_csv(tri_decay_csv)

comsol_t, comsol_s, comsol_peak = trim_to_decay(comsol_t_raw, comsol_s_raw)
pixel_t, pixel_s, pixel_peak = trim_to_decay(pixel_t_raw, pixel_s_raw)
tri_t, tri_s, tri_peak = trim_to_decay(tri_t_raw, tri_s_raw)

comsol_inv = invert_fixed_alpha(comsol_t, comsol_s, name="COMSOL", alpha=NNLS_ALPHA)
pixel_inv = invert_fixed_alpha(pixel_t, pixel_s, name="PNG_pixel", alpha=NNLS_ALPHA)
tri_inv = invert_fixed_alpha(tri_t, tri_s, name="PNG_triangular", alpha=NNLS_ALPHA)

comsol_spec = comsol_inv.spectrum / max(np.max(comsol_inv.spectrum), 1e-30)
pixel_spec = pixel_inv.spectrum / max(np.max(pixel_inv.spectrum), 1e-30)
tri_spec = tri_inv.spectrum / max(np.max(tri_inv.spectrum), 1e-30)

# Optional concentration panel for context.
fig = plt.figure(figsize=(16, 8.5), dpi=160)
gs = fig.add_gridspec(2, 2, width_ratios=[1.05, 1.0], height_ratios=[1, 1], wspace=
ax_img = fig.add_subplot(gs[:, 0])
ax_decay = fig.add_subplot(gs[0, 1])
ax_t2 = fig.add_subplot(gs[1, 1])

if CONCENTRATION_PNG.exists():
    ax_img.imshow(Image.open(CONCENTRATION_PNG).convert("RGB"))
    ax_img.set_title("Concentration field")
else:
    ax_img.imshow(rgb)
    ax_img.set_title("Input phase map")
ax_img.set_axis_off()

ax_decay.plot(comsol_t, normalize(comsol_s), color="#1f77b4", lw=2.0, label="COMSOL")
ax_decay.plot(pixel_t, normalize(pixel_s), color="#d62728", lw=1.8, ls="--", label="PNG_pixel")
ax_decay.plot(tri_t, normalize(tri_s), color="#2ca02c", lw=1.8, ls="-. ", label="PNG_triangular")
ax_decay.set_xlabel("elapsed time after decay peak (ms)")
ax_decay.set_ylabel("normalized signal")
ax_decay.set_title("T2 relaxation decay")
ax_decay.grid(alpha=0.3)
ax_decay.legend()

ax_t2.plot(comsol_inv.t2_bins_ms, comsol_spec, color="#1f77b4", lw=2.0, label="COMSOL")
ax_t2.plot(pixel_inv.t2_bins_ms, pixel_spec, color="#d62728", lw=1.8, ls="--", label="PNG_pixel")
ax_t2.plot(tri_inv.t2_bins_ms, tri_spec, color="#2ca02c", lw=1.8, ls="-. ", label="PNG_triangular")
ax_t2.set_xscale("log")
ax_t2.set_xlim(T2_MIN_MS, T2_MAX_MS)
ax_t2.set_xlabel("T2 (ms)")
ax_t2.set_ylabel("normalized spectral amplitude")
ax_t2.set_title(f"T2 inversion spectra, alpha={NNLS_ALPHA:g}")
ax_t2.grid(alpha=0.3, which="both")
ax_t2.legend()

fig.suptitle(f"{EXP_DIR.name} timestep {Timestep:04d}", fontsize=16)
comparison_png = comparison_dir / f"timestep_{Timestep:04d}_three_way_comparison.png"
fig.savefig(comparison_png, dpi=300, bbox_inches="tight")
plt.show()

t2_export = pd.DataFrame({
    "t2_ms": comsol_inv.t2_bins_ms,

```

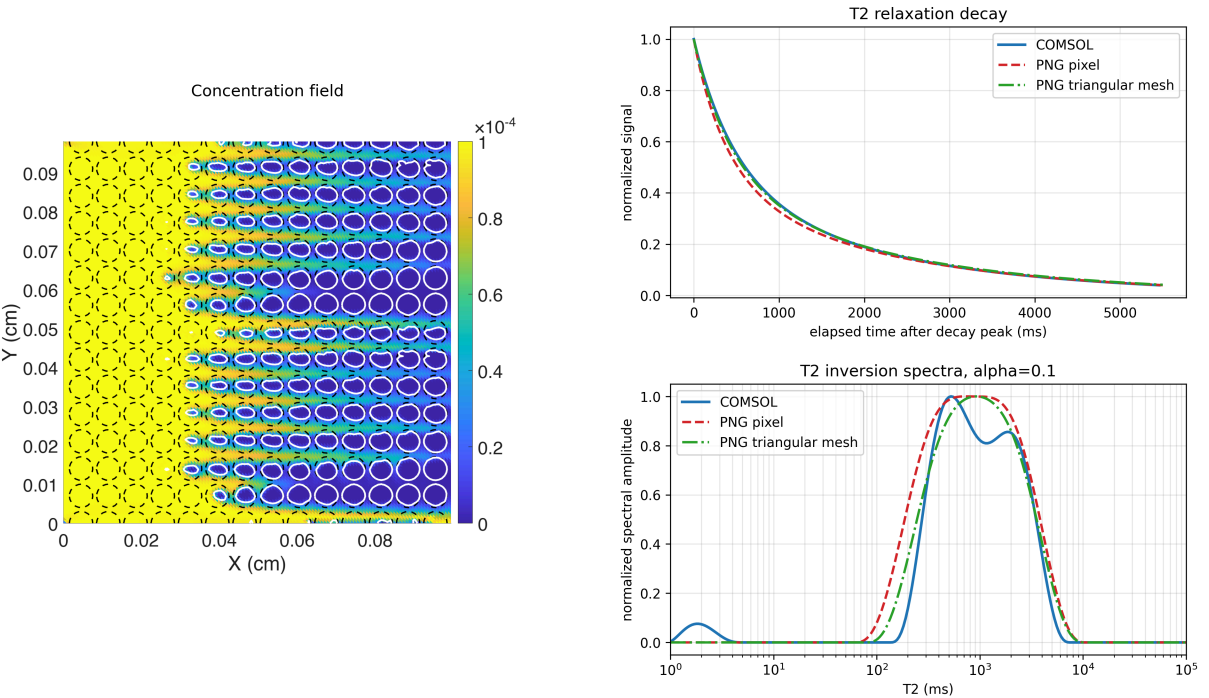
```
"comsol_spectrum_norm": comsol_spec,
"png_pixel_spectrum_norm": pixel_spec,
"png_triangular_spectrum_norm": tri_spec,
})
t2_export.to_csv(comparison_dir / f"timestep_{TIMESTEP:04d}_t2_spectrum_comparison.

summary = pd.DataFrame([
    "timestep": TIMESTEP,
    "comsol_peak_t2_ms": float(comsol_inv.t2_bins_ms[int(np.argmax(comsol_inv.spect
    "png_pixel_peak_t2_ms": float(pixel_inv.t2_bins_ms[int(np.argmax(pixel_inv.spe
    "png_triangular_peak_t2_ms": float(tri_inv.t2_bins_ms[int(np.argmax(tri_inv.spe
    "png_pixel_wall_time_s": pixel_wall_time_s,
    "png_triangular_wall_time_s": tri_wall_time_s,
)])
summary.to_csv(RUN_DIR / "single_timestep_summary.csv", index=False)
display(summary)
display(IPyImage(filename=str(comparison_png)))
```

c:\Users\imgw\conda\envs\ml\Lib\site-packages\openpyxl\styles\stylesheet.py:237: UserWarning: Workbook contains no default style, apply openpyxl's default
warn("Workbook contains no default style, apply openpyxl's default")
C:\Users\imgw\AppData\Local\Temp\ipykernel_60280\2707669420.py:58: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
plt.show()

timestep	comsol_peak_t2_ms	png_pixel_peak_t2_ms	png_triangular_peak_t2_ms	png
0	25	524.403126	734.69788	934.784058

exp_015 timestep 0025



8. Output overview

```
In [16]: print("Run directory:", RUN_DIR)
print("Geometry:", geometry_dir)
print("PNG pixel decay:", pixel_dir)
print("PNG triangular decay:", tri_dir)
print("Comparison:", comparison_dir)
print("Summary CSV:", RUN_DIR / "single_timestep_summary.csv")

# Small timing bar plot for this demo timestep.
timing_fig, ax = plt.subplots(figsize=(5.8, 3.8), dpi=160)
ax.bar(["PNG pixel", "PNG mesh"], [pixel_wall_time_s, tri_wall_time_s], color=["#d6
ax.set_ylabel("wall time (s)")
ax.set_title(f"Demo solve time, timestep {TIMESTEP:04d}")
ax.grid(alpha=0.25, axis="y")
timing_png = RUN_DIR / "single_timestep_timing.png"
timing_fig.savefig(timing_png, dpi=220, bbox_inches="tight")
plt.show()
display(IPyImage(filename=str(timing_png)))
```

```
Run directory: C:\Users\imgw\Documents\Codex\RTSPHEM-main\outputs\rtm_batches\33\exp_015\single_timestep_nmr_demo_t0025_20260528_155335
Geometry: C:\Users\imgw\Documents\Codex\RTSPHEM-main\outputs\rtm_batches\33\exp_015\single_timestep_nmr_demo_t0025_20260528_155335\01_geometry
PNG pixel decay: C:\Users\imgw\Documents\Codex\RTSPHEM-main\outputs\rtm_batches\33\exp_015\single_timestep_nmr_demo_t0025_20260528_155335\02_png_pixel_decay
PNG triangular decay: C:\Users\imgw\Documents\Codex\RTSPHEM-main\outputs\rtm_batches\33\exp_015\single_timestep_nmr_demo_t0025_20260528_155335\03_png_triangular_decay
Comparison: C:\Users\imgw\Documents\Codex\RTSPHEM-main\outputs\rtm_batches\33\exp_015\single_timestep_nmr_demo_t0025_20260528_155335\04_comparison
Summary CSV: C:\Users\imgw\Documents\Codex\RTSPHEM-main\outputs\rtm_batches\33\exp_015\single_timestep_nmr_demo_t0025_20260528_155335\single_timestep_summary.csv
C:\Users\imgw\AppData\Local\Temp\ipykernel_60280\3791287274.py:16: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
plt.show()
```

